

Лабораторна 2
Жила Іван Дмитрович ІМ-42
Контейнеризація

Дослідницька частина

Python Application

1. Створення неоптимізованого образу застосунку

Створюємо неідеальний Dockerfile для застосунку на базі Debian Bookworm, де копіювання всього коду та встановлення залежностей виконуються в одному шарі:

```
FROM python:3.12-bookworm
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN pip install -r requirements.txt
```

```
CMD ["python", "spaceship/app.py"]
```

Перед оцінюванням ефективності завантажуюмо чистий базовий образ, щоб вимірювання часу було об'єктивним:

```
docker pull python:3.12-bookworm
```

Запуск первинного збирання без використання кешу:

```
time docker build --no-cache -t python-lab:test-bookworm .
```

Результати першої збірки:

```
real    0m24.856s
user    0m0.007s
sys     0m0.095s
```

ID	DISK USAGE	CONTENT SIZE	EXTRA
fed14ddf37f7	1.52GB	394MB	

2. Повторна збірка після внесення змін у код

Вносимо зміни у файл `spaceship/app.py`, змушуючи програму виводити ім'я та групу студента:

```
print("Ivan Zhila IM-42")
```

Запускаємо повторну збірку образу з новою версією коду:

```
time docker build -t python-lab:test-bookworm2 .
```

Результати повторної збірки неоптимізованого образу:

```
real    0m19.119s
user    0m0.035s
sys     0m0.116s
```

Висновок: Оскільки інструкція `COPY . .` копіює всі файли проекту разом, будь-яка мінорна зміна у вихідному коді призводить до повної інвалідації кешу для цього шару. Як результат, наступний крок `RUN pip install` повністю скидає кеш і запускається заново. Це неефективно, оскільки важкий процес інсталяції пакетів виконується при кожному білді.

3. Оптимізація структури Dockerfile (Використання шарів)

Для ефективного використання кешування Dockerfile було переписано. Тепер інструкції розташовані від тих, що змінюються найрідше (список залежностей), до тих, що змінюються найчастіше (код програми):

```
FROM python:3.12-bookworm
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
CMD ["python", "spaceship/app.py"]
```

Запуск первинного збирання оптимізованого образу (без кешу):

```
time docker build --no-cache -t python-lab:optimized .
```

Результати першої збірки оптимізованого образу:

```
real    0m28.085s
user    0m0.022s
sys     0m0.076s
```

Після цього у файл коду spaceship/app.py було внесено додаткову зміну в рядок принта та запущено повторне збирання:

```
time docker build -t python-lab:optimized2 .
```

Результати повторної збірки після оптимізації шарів:

```
real    0m5.697s
user    0m0.021s
sys     0m0.055s
```

Аналіз механізму кешування:

Після оптимізації крок COPY requirements.txt . копіює лише файл із бібліотеками. Оскільки цей файл не змінювався, інструкція RUN pip install успішно береться з локального кешу Docker (----> Using cache). Реальна робота системи з генерації нових шарів починається лише на етапі `COPY . .`, де копіюється змінений код програми. Завдяки цьому час повторного збирання після модифікації коду скоротився з 19.119 секунд до 5.697 секунд (чиста економія склала понад 13 секунд на кожному білді, а порівняно з повним чистим розгортанням час зменшився майже в 5 разів — з 28.085 секунд). Такий результат наочно демонструє ефективність правильного розділення шарів у Dockerfile, оскільки тривалий процес мережевого завантаження та інсталяції пакетів повністю пропускається.

4. Дослідження мінімального базового образу (Alpine Linux)

Для максимального зменшення розміру фінального образу було прийнято рішення замінити базовий образ Debian Bookworm на ультралегкий дистрибутив Alpine Linux.

Файл Dockerfile було модифіковано, замінивши перший рядок на:

```
FROM python:3.12-alpine
```

Перед тестуванням було виконано примусове завантаження чистого образу для забезпечення точності вимірювань:

```
docker pull python:3.12-alpine
```

Запуск первинного збирання на базі Alpine (без кешу):

```
time docker build --no-cache -t python-lab:alpine-clean .
```

Результати збірки на Alpine:

```
real    0m17.461s
user    0m0.017s
sys     0m0.084s
```

	ID	DISK USAGE	CONTENT SIZE	EXTRA
an	c237389b564d	123MB	32.3MB	

python-lab-containers-starter-project-python\$ |

Висновок: Перехід на базовий образ Alpine Linux дозволив кардинально зменшити розмір готового контейнера (з 1.52 GB до всього 123 мегабайт). Це досягається завдяки відсутності в Alpine зайвих системних утиліт та використанню оптимізованої бібліотеки musl libc замість стандартної glibc.

5. Тестування важких залежностей (Впровадження NumPy)

Для дослідження поведінки образів при додаванні складних бібліотек, що містять C-розширення, у проєкт було додано пакет `numpy`. У файл `spaceship/routers/api.py` впроваджено новий ендпоінт `/matrix`, який генерує дві випадкові матриці 10x10 та обчислює їхній добуток за допомогою NumPy:

```
from fastapi import APIRouter
```

```
import numpy as np

router = APIRouter()

@router.get("/")
def hello_world() -> dict:
    return {'msg': 'Hello, World!'}

@router.get("/matrix")
def get_matrix():
    matrix_a = np.random.rand(10, 10)
    matrix_b = np.random.rand(10, 10)

    product = np.matmul(matrix_a, matrix_b)

    return {
        "matrix_a": matrix_a.tolist(),
        "matrix_b": matrix_b.tolist(),
        "product": product.tolist()
    }
```

Було проведено порівняльний аналіз часу першої збірки (з чистим кешем) та фінального розміру для обох базових образів:

python:3.12-bookworm:

Запускаємо чисту збірку з NumPy на Debian:

```
time docker build --no-cache -t python-lab:bookworm-numpy .
```

```
real    0m54.033s
user    0m0.135s
sys     0m0.370s
```

Підсумкова вага:

```
docker images python-lab:bookworm-numpy
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
python-lab:bookworm-numpy	fb16d868ba1a	1.64GB	431MB	

python:3.12-alpine:

Запускаємо чисту збірку з NumPy на Alpine:

```
time docker build --no-cache -t python-lab:alpine-numpy .
```

```
real    0m29.847s
user    0m0.037s
sys     0m0.094s
```

Підсумкова вага:

```
docker images python-lab:alpine-numpy
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
python-lab:alpine-numpy	def0cc47a98f	260MB	73.1MB	

Базовий образ	Час першої збірки (real)	Підсумковий розмір образу
python:3.12-bookworm	54.033 s	1.64GB
python:3.12-alpine	29.847 s	260 MB

Порівняльний аналіз результатів:

- Розмір образу: Оптимізований образ на базі Alpine разом із встановленим NumPy важить у 6.4 рази менше за версію на Debian Bookworm (260 MB проти 1.64 GB), що демонструє колосальну економію дискового простору.
- Час збірки: Під час аналізу логів було виявлено, що менеджер пакетів `pip` зміг підібрати `pre-compiled wheel`-пакет спеціально для архітектури Alpine (`musllinux_1_2_x86_64.whl`). Завдяки наявності готового бінарника під системну бібліотеку `musl`, образу Alpine не знадобилося виконувати тривалу компіляцію вихідного коду бібліотеки NumPy з нуля. Як результат, час збірки на Alpine склав всього 29.847 с, що виявилось навіть швидше за завантаження та розпакування аналогічного пакета на Debian Bookworm (54.033 с).

Musl (Alpine) vs glibc (Debian/Ubuntu/etc)

Для дослідження відмінностей у роботі системних бібліотек було створено окрему мережу в Docker:

```
docker network create dns-lab
```

У першому вікні терміналу було запущено локальний DNS-сервер на базі утиліти `dnsmasq`, налаштований резолвити кастомний корпоративний домен `myservice.internal.corp` на IP-адресу 10.0.0.50

У другому вікні терміналу було виконано послідовне тестування резолвінгу короткого імені `myservice.internal` з використанням пошукового суфікса через параметр `--dns-search="corp"` для двох різних дистрибутивів .

1. Результати тестування у контейнері Ubuntu (glibc)

```
ivanadmin@kpi-lab:~$ docker run --rm --network dns-lab \
--dns=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' dns-server) \
--dns-search="corp" \
ubuntu:latest getent hosts myservice.internal
```

```
10.0.0.50    myservice.internal.corp
```

Результат: Контейнер на базі Ubuntu успішно розрезолвив домен і повернув IP 10.0.0.50.

2. Результати тестування у контейнері Alpine (musl libc)

Bash

```
ivanadmin@kpi-lab:~$ docker run --rm --network dns-lab \
--dns=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' dns-server) \
--dns-search="corp" \
alpine:latest getent hosts myservice.internal
```

```
ivanadmin@kpi-lab:~$
```

Результат: Контейнер на базі Alpine повернув порожню відповідь, тобто не зміг знайти хост.

3. Аналіз логів DNS-сервера

```
dnsmasq[1]: query[AAAA] myservice.internal from 172.18.0.3
dnsmasq[1]: forwarded myservice.internal to 127.0.0.11
dnsmasq[1]: reply myservice.internal is NXDOMAIN
dnsmasq[1]: query[AAAA] myservice.internal.corp from 172.18.0.3
dnsmasq[1]: forwarded myservice.internal.corp to 127.0.0.11
dnsmasq[1]: reply myservice.internal.corp is NODATA-IPv6
dnsmasq[1]: query[A] myservice.internal from 172.18.0.3
dnsmasq[1]: cached myservice.internal is NXDOMAIN
dnsmasq[1]: query[A] myservice.internal.corp from 172.18.0.3
dnsmasq[1]: config myservice.internal.corp is 10.0.0.50
dnsmasq[1]: query[AAAA] myservice.internal from 172.18.0.3
dnsmasq[1]: cached myservice.internal is NXDOMAIN
dnsmasq[1]: query[A] myservice.internal from 172.18.0.3
dnsmasq[1]: cached myservice.internal is NXDOMAIN
```

Порівняння результатів (Ubuntu проти Alpine)

Тест чітко показав, що контейнери на різних системах шукають домени по-різному:

⑩ Ubuntu (glibc): Спершу робить запит на коротке ім'я `myservice.internal`.

Сервер відповідає, що такого немає. Тоді Ubuntu дивиться в налаштування `dns-search`, сама дописує в кінець суфікс `.corp` і з другої спроби успішно знаходить повну адресу `myservice.internal.corp` (IP `10.0.0.50`).

⑩ Alpine (musl libc): Робить один запит, отримує відповідь, що хоста немає, і на цьому просто зупиняється. Прапорець `dns-search="corp"` система взагалі проігнорувала.

Чому так вийшло (Різниця між Musl та Glibc)

Вся справа в системних бібліотеках, які відповідають за роботу мережі:

1. В Ubuntu стоїть бібліотека `glibc`. Вона велика і складна. Якщо вона не знаходить хост з першого разу, то починає перебирати всі хвости (суфікси) з файлу конфігурації, поки не знайде правильний.
2. В Alpine стоїть бібліотека `musl libc`. Її максимально урізали, щоб зробити образ легким і швидким. У неї проста логіка: якщо в імені, яке ми шукаємо, є хоча б одна крапка (як у нас `myservice.internal`), вона автоматично вважає це ім'я вже повною адресою. Тому вона робить лише один запит і нічого в кінець не домальовує.

Чому це критично на практиці:

У реальних проєктах на серверах сервіси часто зв'язуються між собою за короткими іменами (наприклад, у конфігах пишуть просто `database.internal`).

Якщо розробник пише і тестує код на звичайній Ubuntu або macOS, усе працюватиме без проблем, бо система сама допише потрібний хвіст. Але якщо цей же проєкт запакувати в Docker-образ на Alpine і випустити на сервер, програма просто "осліпне" і не зможе підключитися до тієї ж бази даних. Причому конфіги будуть абсолютно однаковими, і таку помилку потім дуже важко шукати.

Висновок:

Коли збираємо контейнери на базі Alpine Linux, у конфігах підключень (до баз даних, брокерів чи інших API) треба завжди писати повні адреси (FQDN) — наприклад, `myservice.internal.corp` замість короткого `myservice.internal`. Тоді DNS працюватиме стабільно і передбачувано на будь-якому образі.

Golang Application та Multi-stage builds

1. Звичайна (одноетапна) збірка проєкту

Створюємо стандартний Dockerfile, де в одному контейнері копіюється код, завантажуються залежності та компілюється бінарний файл:

```
FROM golang:1.22-bookworm
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN go build -o main .
```

```
CMD ["/main"]
```

Запуск первинного збирання без кешу:

```
time docker build --no-cache -t go-lab:single .
```

Результати звичайної збірки:

```
real    1m38.954s
user    0m0.084s
sys     0m0.319s
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
go-lab:single	f6da9d9678da	1.33GB	324MB	

```
total 10888
drwxr-xr-x 1 root root    4096 May 17 16:09 .
drwxr-xr-x 1 root root    4096 May 17 16:16 ..
drwxrwxr-x 1 root root    4096 May 17 16:09 .git
-rw-rw-r-- 1 root root   2415 May 17 16:06 .gitignore
-rw-rw-r-- 1 root root     86 May 17 16:08 Dockerfile
-rw-rw-r-- 1 root root   1073 May 17 16:06 README.rst
drwxrwxr-x 2 root root    4096 May 17 16:06 cmd
-rw-rw-r-- 1 root root    181 May 17 16:06 go.mod
-rw-rw-r-- 1 root root  75705 May 17 16:06 go.sum
drwxrwxr-x 2 root root    4096 May 17 16:06 lib
-rwxr-xr-x 1 root root 10817773 May 17 16:09 main
-rw-rw-r-- 1 root root     119 May 17 16:06 main.go
drwxrwxr-x 2 root root    4096 May 17 16:06 templates
```

Аналіз вмісту образу:

Щоб розібратися, з чого складається наш жирний образ, ми подивилися список файлів у робочій папці:

```
docker run --rm go-lab:single ls -la
```

У списку видно сам готовий бінарник ``main`` (він важить близько 10.8 МБ), вихідний код ``main.go``, папки ``cmd``, ``lib``, ``templates`` та файли залежностей ``go.mod`` і ``go.sum``. Також ми перевірили корінь усього контейнера: `docker run --rm go-lab:single ls -la /` Там висвітився повний набір стандартних папок системи Linux Debian (bin, boot, dev, etc, root, usr і т.д.).

Висновок: Для роботи програми всі ці файли не потрібні. ОС Debian та інструменти розробника Go потрібні були лише для того, щоб скомпілювати код. В результаті вони просто так лежать в образі й роздувають його розмір до 1.33 ГБ.

#2. Багатоетапна збірка за допомогою порожнього образу (scratch)

Для оптимізації переписуємо Dockerfile на два етапи. На першому (AS builder) ми компілюємо статичний бінарний файл, а на другому — переносимо тільки його в повністю порожній образ ``FROM scratch``:

```
FROM golang:1.22-bookworm AS builder
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN CGO_ENABLED=0 GOOS=linux go build -o main .
```

```
FROM scratch
```

```
COPY --from=builder /app/main /main
```

```
CMD ["/main"]
```

Запуск збирання:

```
time docker build --no-cache -t go-lab:scratch .
```

Результати збірки зі scratch:

```
real    0m37.086s
user    0m0.018s
sys     0m0.074s
```

Створюємо тимчасовий контейнер (без запуску):

```
docker create --name test-scratch go-lab:scratch
```

Дивимося список файлів всередині цього контейнера:

```
docker export test-scratch | tar -t
```

```
ivanadmin@kpi-lab:~/dep
.dockerenv
dev/
dev/console
dev/pts/
dev/shm/
etc/
etc/hostname
etc/hosts
etc/mtab
etc/resolv.conf
main
proc/
sys/
```

Папки dev, etc, proc створює сам Docker автоматично в момент запуску контейнера. Вони потрібні для базових речей: ізоляції процесів (proc), роботи віртуальних пристроїв (dev) та мережі (etc/hosts і resolv.conf для роботи DNS). Жодного файлу самої операційної системи Linux там немає - з нашого проєкту туди потрапив лише файл main.

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
go-lab:scratch	7c7b63990498	16.7MB	5.98MB	

Відповіді на запитання щодо scratch:

1. Чи достатньо файлів для запуску? Так, цілком достатньо. Мова Go збирає код у самодостатній бінарник, якому для роботи не потрібні зовнішні системні бібліотеки чи інтерпретатори.

2. Чи зручно таким образом користуватися? З точки зору деплою і швидкості - дуже зручно, бо образ важить лічені мегабайти і миттєво завантажується на сервер.

3. Чи зручно виконувати дії/налагодження всередині контейнера? Вкрай незручно. Оскільки образ `scratch` абсолютно порожній, всередині нього немає командного процесора (ні `sh`, ні `bash`), немає утиліт типу `ls`, `cat` чи `ping`. Якщо підключитися всередину запущеного контейнера через `docker exec -it`, система видасть помилку, бо там банально нема чого запускати. Налагоджувати такий контейнер "на гарячу" неможливо.

3. Багатоетапна збірка з використанням образу Distroless

Для вирішення проблем безпеки та сумісності, замість повністю порожнього `scratch` було використано спеціалізований образ від Google — `gcr.io/distroless/static-debian12`. У ньому немає операційної системи, але є кореневі сертифікати та налаштування часу.

Dockerfile було змінено таким чином:

```
FROM golang:1.22-bookworm AS builder
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o main .
```

```
FROM gcr.io/distroless/static-debian12
COPY --from=builder /app/main /main
CMD ["/main"]
```

Запуск збирання:

```
time docker build --no-cache -t go-lab:distroless .
```

Результати збірки з Distroless:

```
real    0m44.590s
user    0m0.038s
sys     0m0.099s
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
go-lab:distroless	fa07a8cc61ac	22.8MB	6.68MB	

Створюємо тимчасовий контейнер:

```
docker create --name test-distroless go-lab:distroless
```

Дивимося файли:

```
docker export test-distroless | tar -t
```

.dockerenv

bin/

boot/

dev/

dev/console

dev/pts/

dev/shm/

etc/

etc/debian_version

etc/default/

etc/dpkg/

etc/dpkg/origins/

etc/dpkg/origins/debian

etc/ethertypes

etc/group

etc/host.conf

etc/hostname
etc/hosts
etc/issue
etc/issue.net
etc/mime.types
etc/mtab
etc/nsswitch.conf
etc/os-release
etc/passwd
etc/profile.d/
etc/protocols
etc/resolv.conf
etc/rpc
etc/services
etc/skel/
etc/ssl/
etc/ssl/certs/
etc/ssl/certs/ca-certificates.crt
etc/update-motd.d/
etc/update-motd.d/10-uname
home/
home/nonroot/
lib/
main
proc/
root/
run/
sbin/

sys/
tmp/
usr/
usr/bin/
usr/games/
usr/include/
usr/lib/
usr/lib/os-release
usr/sbin/
usr/share/
usr/share/base-files/
usr/share/base-files/dot.bashrc
usr/share/base-files/dot.profile
usr/share/base-files/dot.profile.md5sums
usr/share/base-files/info.dir
usr/share/base-files/motd
usr/share/base-files/profile
usr/share/base-files/profile.md5sums
usr/share/base-files/staff-group-for-usr-local
usr/share/bug/
usr/share/bug/media-types/
usr/share/bug/media-types/presubj
usr/share/common-licenses/
usr/share/common-licenses/Apache-2.0
usr/share/common-licenses/Artistic
usr/share/common-licenses/BSD
usr/share/common-licenses/CC0-1.0
usr/share/common-licenses/GFDL

usr/share/common-licenses/GFDL-1.2
usr/share/common-licenses/GFDL-1.3
usr/share/common-licenses/GPL
usr/share/common-licenses/GPL-1
usr/share/common-licenses/GPL-2
usr/share/common-licenses/GPL-3
usr/share/common-licenses/LGPL
usr/share/common-licenses/LGPL-2
usr/share/common-licenses/LGPL-2.1
usr/share/common-licenses/LGPL-3
usr/share/common-licenses/MPL-1.1
usr/share/common-licenses/MPL-2.0
usr/share/dict/
usr/share/doc/
usr/share/doc/base-files/
usr/share/doc/base-files/FAQ
usr/share/doc/base-files/README
usr/share/doc/base-files/README.FHS
usr/share/doc/base-files/changelog.gz
usr/share/doc/base-files/copyright
usr/share/doc/ca-certificates/
usr/share/doc/ca-certificates/copyright
usr/share/doc/media-types/
usr/share/doc/media-types/changelog.gz
usr/share/doc/media-types/copyright
usr/share/doc/netbase/
usr/share/doc/netbase/changelog.gz
usr/share/doc/netbase/copyright

usr/share/doc/tzdata/
usr/share/doc/tzdata/README.Debian
usr/share/doc/tzdata/changelog.Debian.gz
usr/share/doc/tzdata/changelog.gz
usr/share/doc/tzdata/copyright
usr/share/info/
usr/share/lintian/
usr/share/lintian/overrides/
usr/share/lintian/overrides/base-files
usr/share/lintian/overrides/tzdata
usr/share/man/
usr/share/misc/
usr/share/zoneinfo/
usr/share/zoneinfo/Africa/
usr/share/zoneinfo/Africa/Abidjan
usr/share/zoneinfo/Africa/Accra
usr/share/zoneinfo/Africa/Addis_Ababa

і т.д

Аналіз вмісту образу Distroless:

По логах чітко видно, що окрім нашого бінарника main, всередині з'явилися дуже важливі системні файли, яких не було в scratch:

1. etc/ssl/certs/ca-certificates.crt — кореневі сертифікати безпеки. Без них наша програма не змогла б робити безпечні запити в інтернет (наприклад, викликати сторонні API по HTTPS).
2. usr/share/zoneinfo/ — величезна база даних таймзон (від Африки до Європи/Київ). Вона потрібна, щоб програма всередині контейнера правильно розуміла локальний час.
3. etc/passwd та home/nonroot/ — налаштування користувачів. Контейнери Distroless за замовчуванням дозволяють запускати софт не від імені суперкористувача (root), а від безпечного користувача nonroot.

При цьому в образі немає жодних утиліт Лінуksа, немає компіляторів і немає командної оболонки (sh/bash).

Порівняння з минулими етапами:

- Розмір образу: Він вийшов зовсім трохи більшим за scratch (десь на 6 Мегабайт: 22.8 МБ проти 16.7 МБ), але все одно залишається крихітним порівняно з гігабайтним Debian Bookworm (який у нас заважив аж 1.33 ГБ).

- Зручність та адміністрування: У плані адміністрування тут ситуація така ж, як і зі scratch. Усередині Distroless немає командної оболонки sh або bash, тому виконати docker exec і зайти всередину контейнера для перевірки файлів неможливо.

Висновок: Образи Distroless є найкращим вибором для продакшену. Вони дають таку ж екологічність та мінімальну вагу, як і scratch, але додають критично важливі для реальних програм речі (наприклад, SSL-сертифікати для роботи з інтернетом та налаштування таймзон), при цьому не засмічуючи контейнер зайвим софтом.